

Code-Layout, Readability and Reusability

Date	01 July 2026
Team ID	N/A
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used Throughout the code	Yes	Uniform indentation and formatting followed across all Python files.
2	Proper File Structure	Files and folders are logically organized	Yes	Project organized into app , frontend , services , routers , models , tests , and configuration files.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Uses meaningful names such as <code>event_details</code> , <code>conversation_history</code> , <code>response</code> , and <code>fact_result</code> .
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as <code>generate_conversation()</code> , <code>generate_self_intro()</code> , <code>generate_tips()</code> , <code>check_facts()</code> , and <code>save_history()</code> clearly describe their functionality.
5	Code Comments	Inline and block comments explain log	Partial	Important functions are documented with comments where required.
6	Modular Design	Code is split into reusable functions/modules	Yes	Conversation generation, self-introduction, networking tips, fact checking, history management, and API routes are implemented as separate modules.

7	No Redundant Code	Duplicate or unused code is removed	Yes	Reusable service modules minimize duplicate code and improve maintainability.
8	Error Handling	Exceptions and errors are handled carefully	Yes	FastAPI validation, API exception handling, and input validation improve reliability.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Conversation Generation Service	Python, Groq API	AI conversation starter generation	High
2	Self Introduction Service	Python, Groq API	Personalized self-introduction generation	High
3	Networking Tips Service	Python, Groq API	Networking suggestions	High
4	Fact Checker Service	Python, Wikipedia API	Fact verification	High
5	History Logger Service	Python, SQLite	Conversation history storage and retrieval	High
6	FastAPI Routes	FastAPI	Backend API endpoints	High
7	Streamlit UI Components	Streamlit	User Interface	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Well-organized project with modular architecture.
Readability	5	Easy-to-read code with meaningful naming conventions.
Reusability	5	Core services are reusable across multiple workflows.
Documentation / Comments	4	Adequate comments and project documentation provided.
Overall Score	4.8/5	Clean, modular, maintainable, and scalable codebase suitable for future enhancements.